



ESCUELA DE EDUCACION SECUNDARIA TECNICA N° 5

“2 DE ABRIL” – TEMPERLEY – BUENOS AIRES

Gimbal ejes XY

MATERIA: PROYECTO Y DISEÑO ELECTRONICO

AUTORES:

Ignacio Alfredo Sánchez

Lautaro Nicolás Reyes

Alex Fabian Pereira

Matías Coronel

NOMBRES Y MAIL:

PROFESOR BERNASSANI

lautyreyes2007@gmail.com

Reyes

21pereiraalex@gmail.com

Pereira

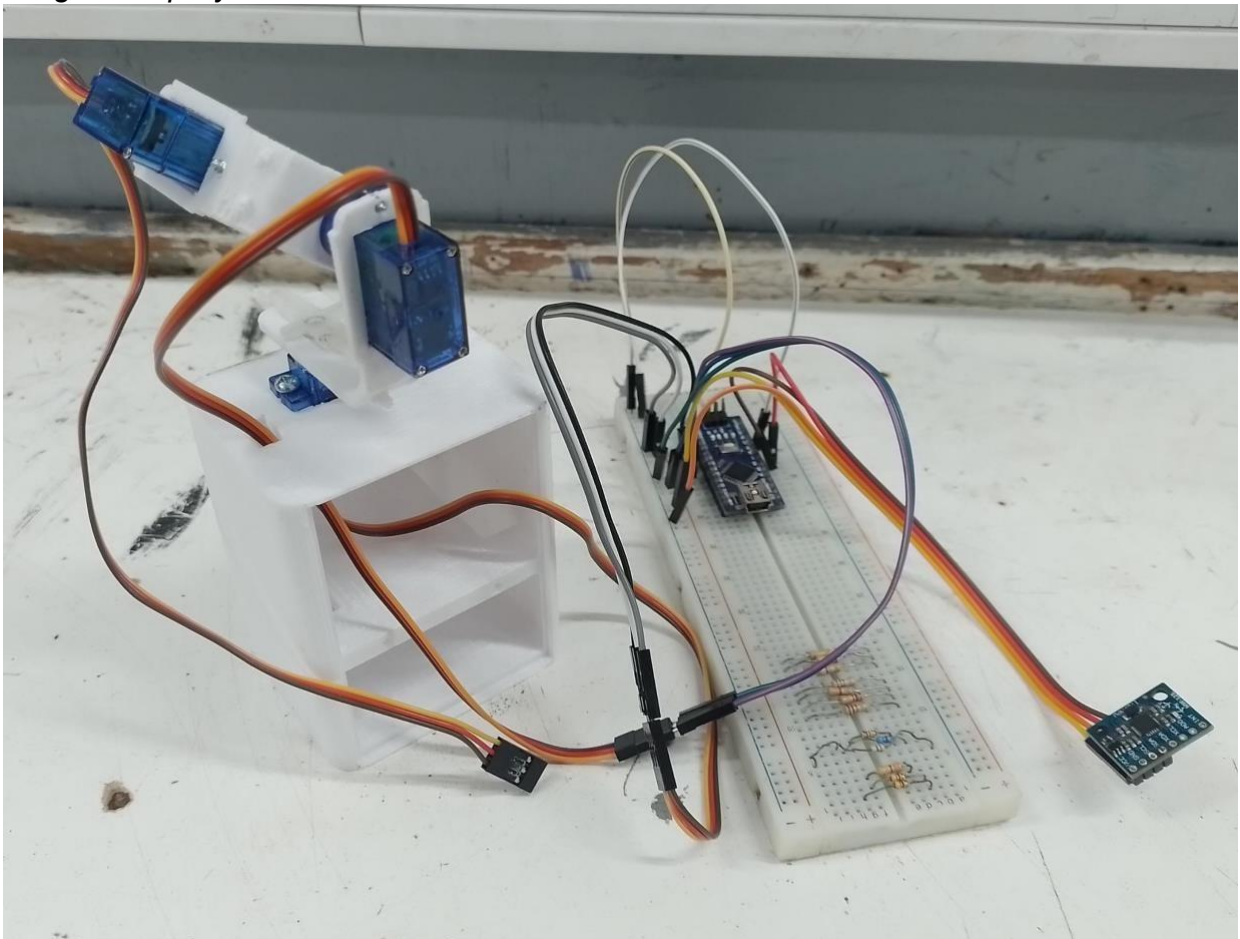
Índice

- Resumen
- Introducción teórica
- Software
- Análisis económico
- Conclusiones
- GitHub

Resumen

El presente proyecto consiste en el desarrollo de un gimbal de dos ejes controlado mediante un sistema PID. El objetivo principal es estabilizar una plataforma utilizando un sensor inercial MPU6050 y dos microservos SG90, procesados por una placa Arduino Nano. El sistema permite compensar movimientos no deseados manteniendo la orientación estable.

Imágen del proyecto



Introducción teórica

Un gimbal es un sistema de estabilización que permite mantener una orientación constante frente a movimientos. Para este proyecto se utiliza un sensor MPU6050, el cual integra un acelerómetro y un giroscopio de tres ejes.

El control del sistema se realiza mediante un controlador PID (Proporcional, Integral y Derivativo), el cual corrige los errores de posición calculando una señal de control adecuada para los actuadores.

El sistema mide continuamente la orientación de la plataforma mediante el MPU6050. A partir de los datos del acelerómetro y el giroscopio, se calculan los ángulos de inclinación utilizando un filtro complementario.

Luego, estos ángulos son comparados con los valores deseados, generando un error que es procesado por el controlador PID para cada eje.

El hardware del proyecto está compuesto por una placa Arduino Nano, un sensor MPU6050 y dos microservos SG90.

El MPU6050 se comunica con el Arduino mediante el protocolo I2C, mientras que los servos reciben señales PWM desde pines digitales. La alimentación se realiza desde la fuente del microcontrolador.

Descripción del software

El programa se encarga de estabilizar el gimbal en dos ejes mediante el uso de un sensor MPU6050 y un controlador PID. El sensor mide la orientación del sistema a través de un acelerómetro y un giroscopio, cuyos datos son procesados para obtener los ángulos de inclinación.

Para mejorar la precisión, se utiliza un filtro complementario que combina la información del acelerómetro y del giroscopio, logrando ángulos estables en los ejes Roll y Pitch. Estos ángulos se comparan con un valor de referencia de 0° , generando un error que es procesado por el controlador PID.

El controlador PID calcula la corrección necesaria para cada eje mediante las acciones proporcional, integral y derivativa. La salida del PID se limita a valores seguros y se aplica a los servomotores, los cuales ajustan su posición para compensar las inclinaciones detectadas.

El proceso se repite continuamente dentro del bucle principal del programa, permitiendo la estabilización automática y en tiempo real del gimbal.

```

#include "I2Cdev.h"
#include "MPU6050.h"
#include <Wire.h>
#include <Servo.h>

MPU6050 mpu;

// Servos
Servo servoRoll;
Servo servoPitch;

// MPU RAW
int ax, ay, az;
int gx, gy, gz;

// Tiempo
unsigned long tiempo_prev;
float dt;

// Ángulos
float ang_x, ang_y;
float ang_x_prev, ang_y_prev;

/* -- Config de PID -- */
float roll_setpoint = 0; // Setpoints
float pitch_setpoint = 0;

float roll_error, pitch_error; // Errores
float roll_prev_error = 0;
float pitch_prev_error = 0;

float roll_p, roll_i = 0, roll_d; // PID términos
float pitch_p, pitch_i = 0, pitch_d;

float Kp = 2.0; // Constantes PID
float Ki = 0.05;
float Kd = 0.01;

float roll_PID, pitch_PID; // Salida PID

void lecturaMPU()
{
    // Lectura de giroscopo y acelerometro
    mpu.getAcceleration(&ax, &ay, &az);
    mpu.getRotation(&gx, &gy, &gz);
}

```

```

dt = (millis()-tiempo_prev)/1000.0;
tiempo_prev=millis();

//Calcular los ángulos con acelerometro
float accel_ang_x=atan(ay/sqrt(pow(ax,2) + pow(az,2)))*(180.0/3.14);
float accel_ang_y=atan(-ax/sqrt(pow(ay,2) + pow(az,2)))*(180.0/3.14);

//Calcular angulo de rotación con giroscopo y filtro complemento
ang_x = 0.98*(ang_x_prev+(gx/131)*dt) + 0.02*accel_ang_x;
ang_y = 0.98*(ang_y_prev+(gy/131)*dt) + 0.02*accel_ang_y;

ang_x_prev=ang_x;
ang_y_prev=ang_y;
}

void calcularPID() {
  // Error
  roll_error = ang_y - roll_setpoint;
  pitch_error = ang_x - pitch_setpoint;

  // Proporcional
  roll_p = Kp * roll_error;
  pitch_p = Kp * pitch_error;

  // Integral
  if (roll_error > -3 && roll_error < 3)
    roll_i += Ki * roll_error;

  if (pitch_error > -3 && pitch_error < 3)
    pitch_i += Ki * pitch_error;

  // Derivativo
  roll_d = Kd * ((roll_error - roll_prev_error) / dt);
  pitch_d = Kd * ((pitch_error - pitch_prev_error) / dt);

  // PID total
  roll_PID = roll_p + roll_i + roll_d;
  pitch_PID = pitch_p + pitch_i + pitch_d;

  // Limitar salida
  roll_PID = constrain(roll_PID, -90, 90);
  pitch_PID = constrain(pitch_PID, -90, 90);

  roll_prev_error = roll_error;
  pitch_prev_error = pitch_error;
}

```

```

}

void moverServos() {
  servoRoll.write(90 - roll_PID);
  servoPitch.write(90 + pitch_PID);
}

void setup() {
  Serial.begin(115200);
  Wire.begin();
  mpu.initialize();

  servoRoll.attach(3);
  servoPitch.attach(5);

  tiempo_prev = millis();

  if (mpu.testConnection())
    Serial.println("MPU OK");
  else
    Serial.println("Error MPU");
}

void loop() {
  lecturaMPU();
  calcularPID();
  moverServos();

  //Mostrar los angulos separadas por un [tab]
  Serial.print("Rotacion en X: ");
  Serial.print(ang_x);
  Serial.print("\tRotacion en Y: ");
  Serial.println(ang_y);
}

```

Análisis económico

Estimación del costo total:

- Arduino Nano: \$12.000
 - Sensor MPU6050: \$5.000
 - Dos microservomotores SG90: \$5.000
 - Material para impresión 3D: \$7.000
- Total aproximado: **\$34.000**

Conclusiones

Se logró implementar un sistema de estabilización funcional mediante control PID. El proyecto cumple con los objetivos planteados y permite futuras mejoras como la optimización de los parámetros PID o la incorporación de un eje adicional.

Github

En este link encontrará todo lo necesario para la realización del proyecto:

<https://github.com/Corviknight07/Gimball.git>